HOTEL BILLING AND CHECK-OUT SYSTEM

MINI PROJECT REPORT

Submitted by

MUKESH P 2117240070193

in partial fulfillment for the award of the

degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

RAJALAKSHMI INSTITUTE OF TECHNOLOGY

KUTHAMBAKKAM, CHENNAI - 600 124

ANNA UNIVERSITY: CHENNAI 600 025

NOV/DEC - 2025

ANNA UNIVERSITY: CHENNAI - 600 025

BONAFIDE CERTIFICATE

Certified that this Mini Project report **"HOTEL BILLING AND CHECK-OUT SYSTEM"** is the Bonafide work of **"MUKESH P(2117240070193)"**, who carried out the project work under my supervision.

SIGNATURE

HEAD OF THE DEPARTMENT

Dr. N. KANAGAVALLI, Ph.D

Assistant Professor,

Artificial Intelligence and Data Science,

Rajalakshmi Institute of Technology,

Chennai – 600 124

SIGNATURE

SUPERVISOR

Mrs. R. KALAIYARASI, M.E

Assistant Professor,

Artificial Intelligence and Data Science,

Rajalakshmi Institute of Technology, Chennai – 600 124

Submitted for the Mini Project viva-voce held on ……..

INTERNAL EXAMINER

EXTERNAL EXAMINER

PROJECT DESCRIPTION

The Hotel Billing and Check-Out System is a Java-based mini project developed using Object-Oriented Programming (OOP) principles. The primary aim of this project is to automate hotel billing and check-out operations to reduce manual effort, eliminate calculation errors, and streamline customer management in small and medium-sized hotel establishments. It provides a simple, effective, and secure way for hotel staff to manage guests, record their stay details, and generate accurate bills with minimal human intervention.

This system features both a console-based interface and a JavaFX graphical user interface (GUI), ensuring an interactive and user-friendly experience. The application allows hotel staff to enter essential customer information such as name, contact number, stay duration, and room type. Additional services like food, laundry, and Wi-Fi can also be selected by the staff, and the system automatically computes the total bill based on the room rate, duration of stay, and selected services.

The backend of the project is implemented using MySQL, and database connectivity is handled through JDBC (Java Database Connectivity). This ensures reliable data storage, retrieval, and update functionalities for billing records. Administrative users can view, modify, and manage historical customer data, thereby improving accuracy and efficiency in hotel record maintenance.

By integrating Java programming with database management and GUI technologies, this project demonstrates a practical application of OOP concepts such as encapsulation, inheritance, and abstraction. The system can be further enhanced with modules for online payment integration, room availability tracking, advanced reporting, and user authentication. Overall, the Hotel Billing and Check-Out System provides a robust, scalable, and efficient solution for automating hotel management and billing operations.

INTRODUCTION

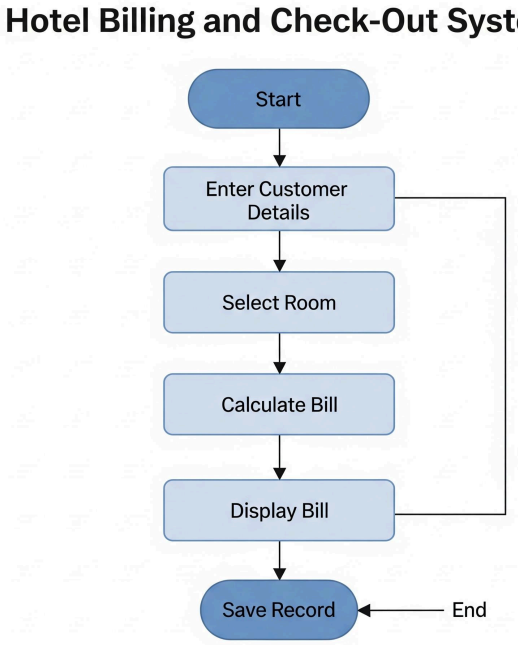
The Hotel Billing and Check-Out System is a Java-based mini project designed to automate hotel operations. The system simplifies the process of calculating bills, recording customer details, and managing check-outs efficiently. It reduces human errors and saves time by generating bills automatically. The project follows Object-Oriented Programming (OOP) principles, which improve modularity and code reusability. Using JavaFX, the system provides a graphical interface for easy user interaction. A MySQL database is integrated to store and retrieve customer and billing records. The system enables hotel staff to manage bookings and services efficiently. It also supports additional features like food, laundry, and Wi-Fi billing. This project helps small and medium hotels digitalize their operations. Overall, it enhances accuracy, reliability, and customer satisfaction.

OBJECTIVES

The main objectives of the project are:

- To automate the billing and check-out process for hotel guests.
- To maintain accurate and easily accessible records of customer information and room occupancy.
- To minimize human errors associated with manual calculation of charges.
- To develop a robust and user-friendly Java application using Object-Oriented Programming principles.
- To enhance efficiency in hotel operations by providing a streamlined management system.

FLOW CHART



PROJECT STRUCTURE

```
├── src/
│   ├── Miniproject/
│   │   ├── Customer.java
│   │   ├── Room.java
│   │   ├── Services.java
│   │   ├── HotelBillingSystem.java
│   │   ├── HotelBillingFx.java
│   │   └── DBConnection.java
├── lib/
│   ├── mysql-connector.jar
│   └── javafx-sdk-21/
├── database/
│   └── hotel_db.sql
└── README.txt
```

CODE

1.Customer.java:

```
package Miniproject;
public class Customer {
   private String name;
   private int days;
   private int phno;
   private Room room;
   private Services services;
     public Customer(String name, int phno, int days, Room room, Services
services) {
       this.name = name;
       this.days = days;
       this.room = room;
       this.phno = phno;
       this.services = services;
   }
   public double generateBill() {
       double roomCost = room.calculateCharge(days);
       double serviceCost = services.totalServiceCost();
       return roomCost + serviceCost;
   }
   public void printBill() {
       System.out.println("------ Hotel Bill ------");
       System.out.println("Customer: " + name);
       System.out.println("Phone: " + phno);
       System.out.println("Room Type: " + room.getRoomType());
       System.out.println("Days Stayed: " + days);
       System.out.println("Room Charges: " + room.calculateCharge(days));
       System.out.println("Service Charges: " + services.totalServiceCost());
       System.out.println("Total Bill: " + generateBill());
       System.out.println("------------------------");
   }
}
```

2.Room.java:

```
package Miniproject;
public abstract class Room {
    protected double rate;
    public abstract double calculateCharge(int days);
    public abstract String getRoomType();
}
class StandardRoom extends Room {
    public StandardRoom() {
        rate = 1000;
    }
    @Override
    public double calculateCharge(int days) {
        return rate * days;
    }
    @Override
    public String getRoomType() {
        return "Standard";
    }
}
class DeluxeRoom extends Room {
    public DeluxeRoom() {
        rate = 2000;
    }
    @Override
    public double calculateCharge(int days) {
        return rate * days;
    }
    @Override
    public String getRoomType() {
        return "Deluxe";
    }
}
class SuiteRoom extends Room {
    public SuiteRoom() {
        rate = 4000;
    }
    @Override
```

```
    public double calculateCharge(int days) {
        return rate * days;
    }
    @Override
    public String getRoomType() {
        return "Suite";
    }
}
```

3.Services:

```
package Miniproject;
public class Services {
    private boolean food, laundry, wifi;
    public Services(boolean food, boolean laundry, boolean wifi) {
        this.food = food;
        this.laundry = laundry;
        this.wifi = wifi;
    }
    public double totalServiceCost() {
        double cost = 0;
        if (food)
            cost += 500;
        if (laundry)
            cost += 300;
        if (wifi)
            cost += 200;
        return cost;
    }
}
```

4.HotelBillingSystem.java:

```
package Miniproject;
import java.util.*;
public class HotelBillingSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter Customer Name: ");
        String name = sc.nextLine();
        System.out.print("Enter phone number: ");
        int phno = sc.nextInt();
        System.out.print("Enter Days of Stay: ");
        int days = sc.nextInt();
        System.out.println("Choose Room Type: 1.Standard 2.Deluxe 3.Suite");
        int choice = sc.nextInt();
        Room room;
        switch (choice) {
            case 1:
                room = new StandardRoom();
                break;
            case 2:
                room = new DeluxeRoom();
                break;
            case 3:
                room = new SuiteRoom();
                break;
            default:
                room = new StandardRoom();
        }
        System.out.print("Need Food service? (true/false): ");
        boolean food = sc.nextBoolean();
        System.out.print("Need Laundry service? (true/false): ");
        boolean laundry = sc.nextBoolean();
        System.out.print("Need WiFi service? (true/false): ");
        boolean wifi = sc.nextBoolean();
        Services services = new Services(food, laundry, wifi);
        Customer customer = new Customer(name, phno, days, room, services);
        customer.printBill();
        sc.close();
    }
}
```

5.HotelBillingFx.java:

```
package Miniproject;
import javafx.application.Application;
```

```
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.stage.Stage;
import java.sql.Connection;
import java.sql.PreparedStatement;
public class HotelBillingFx extends Application {
    @Override
    public void start(Stage primaryStage) {
        TextField nameField = new TextField();
        TextField phoneField = new TextField();
        TextField daysField = new TextField();
        ComboBox<String> roomBox = new ComboBox<>();
        roomBox.getItems().addAll("Standard", "Deluxe", "Suite");
        roomBox.setValue("Standard");
        CheckBox foodBox = new CheckBox("Food Service (₹500)");
        CheckBox laundryBox = new CheckBox("Laundry (₹300)");
        CheckBox wifiBox = new CheckBox("WiFi (₹200)");
        Button billButton = new Button("Generate Bill");
        TextArea billArea = new TextArea();
        billArea.setEditable(false);
        billArea.setPrefHeight(250);
        VBox root = new VBox(10);
        root.setPadding(new Insets(15));
        root.getChildren().addAll(
            new Label("Customer Name:"), nameField,
            new Label("Phone Number:"), phoneField,
            new Label("Days of Stay:"), daysField,
            new Label("Room Type:"), roomBox,
            new Label("Additional Services:"),
            foodBox, laundryBox, wifiBox,
            billButton, billArea
        );
        billButton.setOnAction(e -> {
            try {
                String name = nameField.getText();
                String phone = phoneField.getText();
```

```
            int days = Integer.parseInt(daysField.getText());
            String roomType = roomBox.getValue();
            Room room;
            switch (roomType) {
               case "Deluxe": room = new DeluxeRoom(); break;
               case "Suite": room = new SuiteRoom(); break;
               default: room = new StandardRoom();
            }
            Services services = new Services(
               foodBox.isSelected(),
               laundryBox.isSelected(),
               wifiBox.isSelected()
            );
                    Customer customer = new Customer(name, 0, days, room,
services);
            double roomCharge = room.calculateCharge(days);
            double serviceCharge = services.totalServiceCost();
            double total = customer.generateBill();
            String billText = "------ Hotel Bill ------\n"
                  + "Customer: " + name + "\n"
                  + "Phone: " + phone + "\n"
                  + "Room Type: " + room.getRoomType() + "\n"
                  + "Days Stayed: " + days + "\n"
                  + "Room Charges: ₹" + roomCharge + "\n"
                  + "Service Charges: ₹" + serviceCharge + "\n"
                  + "Total Bill: ₹" + total + "\n"
                  + "------------------------";
            billArea.setText(billText);
            try (Connection con = DBConnection.getConnection()) {
                     String sql = "INSERT INTO bills (customer_name, phone,
room_type, days, room_charge, service_charge, total) VALUES (?, ?, ?, ?, ?,
?, ?)";
               PreparedStatement ps = con.prepareStatement(sql);
               ps.setString(1, name);
               ps.setString(2, phone);
               ps.setString(3, roomType);
               ps.setInt(4, days);
               ps.setDouble(5, roomCharge);
```

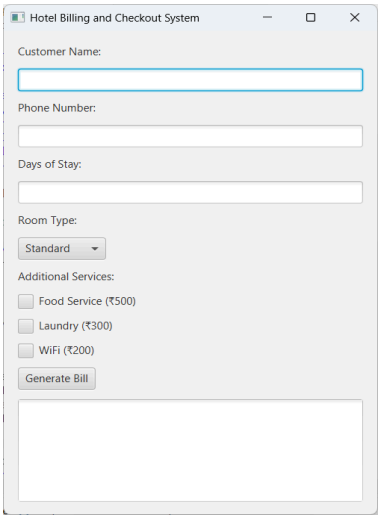
```
                ps.setDouble(6, serviceCharge);
                ps.setDouble(7, total);
                ps.executeUpdate();
                        billArea.appendText("\n\n Record Saved to Database
Successfully!");
            } catch (Exception ex) {
                billArea.appendText("\n\n Database Error: " + ex.getMessage());
            }
        } catch (NumberFormatException ex) {
            billArea.setText("Error: Please enter valid number for Days.");
        } catch (Exception ex) {
            billArea.setText("Error: " + ex.getMessage());
        }
    });
    Scene scene = new Scene(root, 400, 550);
    primaryStage.setTitle("Hotel Billing and Checkout System");
    primaryStage.setScene(scene);
    primaryStage.show();
  }
  public static void main(String[] args) {
    launch(args);
  }
}
```

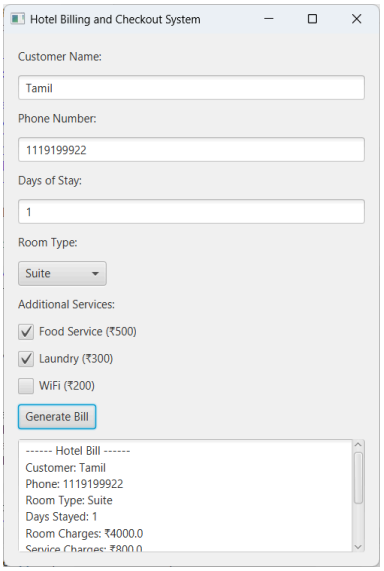
6.DBConnection.java:

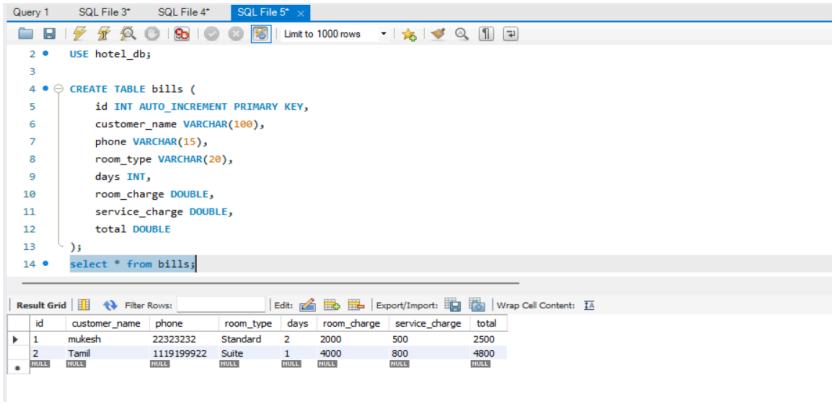
```
package Miniproject;
import java.sql.*;
public class DBConnection {
  private static final String URL = "jdbc:mysql://localhost:3306/hotel_db";
  private static final String USER = "root";
  private static final String PASS = "232007";
  public static Connection getConnection() {
    Connection con = null;
    try {
      con = DriverManager.getConnection(URL, USER, PASS);
    } catch (SQLException e) {
      System.out.println("Database Connection Failed: " + e.getMessage());
```

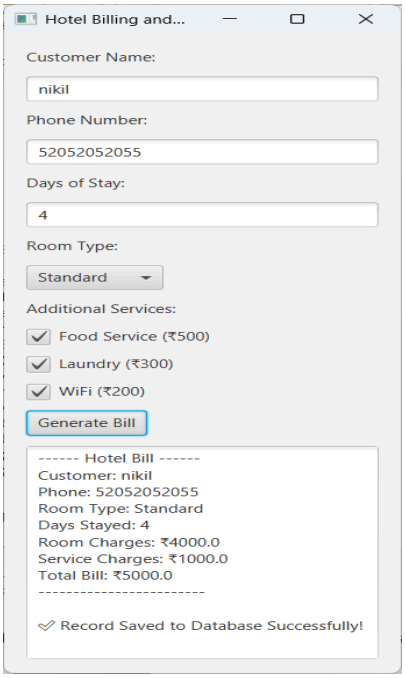
```
        }
        return con;
    }
}
```

OUTPUT SCREENSHOTS









CONCLUSION & FUTURE WORK

The Hotel Billing and Check-Out System successfully automates hotel billing, data storage, and check-out operations using Java OOP concepts. It provides an efficient way to manage customer information and generate accurate bills. The system improves accuracy, reduces manual workload, and enhances data organization. Integration with MySQL ensures data persistence and reliability. The user interface built with JavaFX provides easy interaction and clear output. In the future, the system can include online payment integration and real-time room availability tracking. Additional features like revenue analytics and multi-user authentication can enhance performance. The project demonstrates strong OOP design principles and database connectivity. Overall, it offers a scalable and practical solution for modern hotel management.